Yenepoya (Deemed to be University).

Project –High Level Design

on

Zero-Day Scanner for Supply Chain Web Portals

Student Name:

Ashin Mohammed KT

Industry Mentor: Mr. Shashank Korram **Guided by: Mr. Vikyath**

Table of Contents

1. Introduction 3
   1.1 Scope of the document 3
   1.2 Intended Audience 3
   1.3 System overview 3

2. System Design 4
   2.1. Application Design 4
   2.2. Process Flow 4
   2.3. Information Flow 5
   2.4. Components Design 6
   2.5. Key Design Considerations 7
   2.6. API Catalogue 7

3. Data Design 8
   3.1. Data Model 8
   3.2. Data Access Mechanism 8
   3.3. Data Retention Policies 8
   3.4. Data Migration 9

4. Interfaces 10

5. State and Session Management 11

6. Caching 11

7. Non-Functional Requirements 11
   5.1. SecurityAspects 11
   5.2 Performance Aspects 11

8. References 12

High-Level Design (HLD):Zero-Day Scanner for Supply Chain Web Portals

1. INTRODUCTION

1.1. SCOPE OF THE DOCUMENT

This High-Level Design (HLD) document describes the overall architectural design, component interactions, data flows, and technical decisions for the ****Zero-Day Scanner for Supply Chain Web Portal Security**** system. It provides a blueprint for developers, evaluators, and academic reviewers to understand how the system is structured and how its components interact.

This document covers:

- The overall system architecture and technology stack.
- The design of the web-based frontend (HTML/CSS/JS) and the Python/Flask backend.
- The link-crawling, status-checking, and reporting pipeline.
- Interfaces between system components.
- Non-functional requirements such as security and performance.

1.2. INTENDED AUDIENCE

This document is intended for the following readers:

- Academic evaluators and faculty use this document to evaluate the system design quality.
- The development team or student uses it to guide implementation and understand system components.
- Cybersecurity researchers use it to study the scanning methodology and threat model.
- Project reviewers use it to verify that the system meets the defined objectives.

1.3. SYSTEM OVERVIEW

The **Zero-Day Scanner** is a web-based security tool designed to analyze e-commerce and supply chain portals by detecting broken, redirected, and potentially risky links.

These portals contain many internal and external links, which can lead to serious issues such as:

- Data theft through malicious external links
- Service disruption due to broken links
- Supply chain attacks via redirects

The tool works by scanning a user-provided URL, extracting all links, and checking their HTTP status in parallel. It categorizes results as:

- **Working** (2xx responses)
- **Broken** (4xx/5xx or errors)
- **Redirect** (3xx responses)
- **External** (different domain links)

The system is built using **Python with Flask (backend)** and **HTML, CSS, and JavaScript (frontend)**, running on a local server.

2. SYSTEM DESIGN

2.1. APPLICATION DESIGN.

The application follows a two-tier MVC-inspired architecture where the frontend (browser) communicates with the backend using HTTP POST requests. The client side consists of index.html for user input, results.html for displaying scan results, and a shared style.css for styling, all rendered through Flask using Jinja2 templates. The backend is developed using Python and Flask, where app.py manages routing and handles user requests.

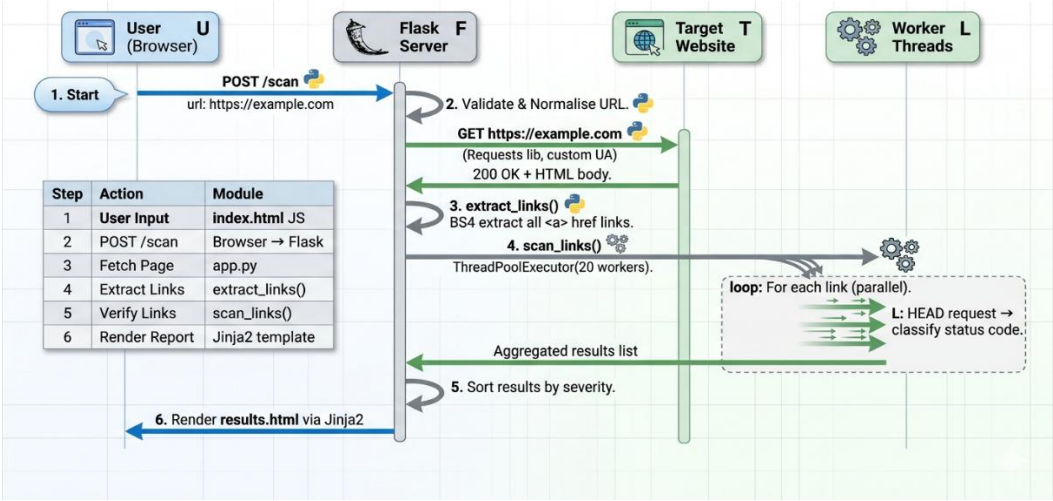
When a user submits a URL, the backend fetches the webpage using the fetch_page() function, extracts all hyperlinks using extract_links() with BeautifulSoup, and then checks each link's status in parallel using scan_links() with ThreadPoolExecutor. The processed results are then sent back to the frontend and displayed in a structured report format.

The system uses Python as the core programming language, Flask as the web framework, BeautifulSoup for HTML parsing, Requests for HTTP communication, and standard concurrency tools for faster link checking, while the frontend is built using HTML, CSS, and JavaScript.

2.2. PROCESS FLOW

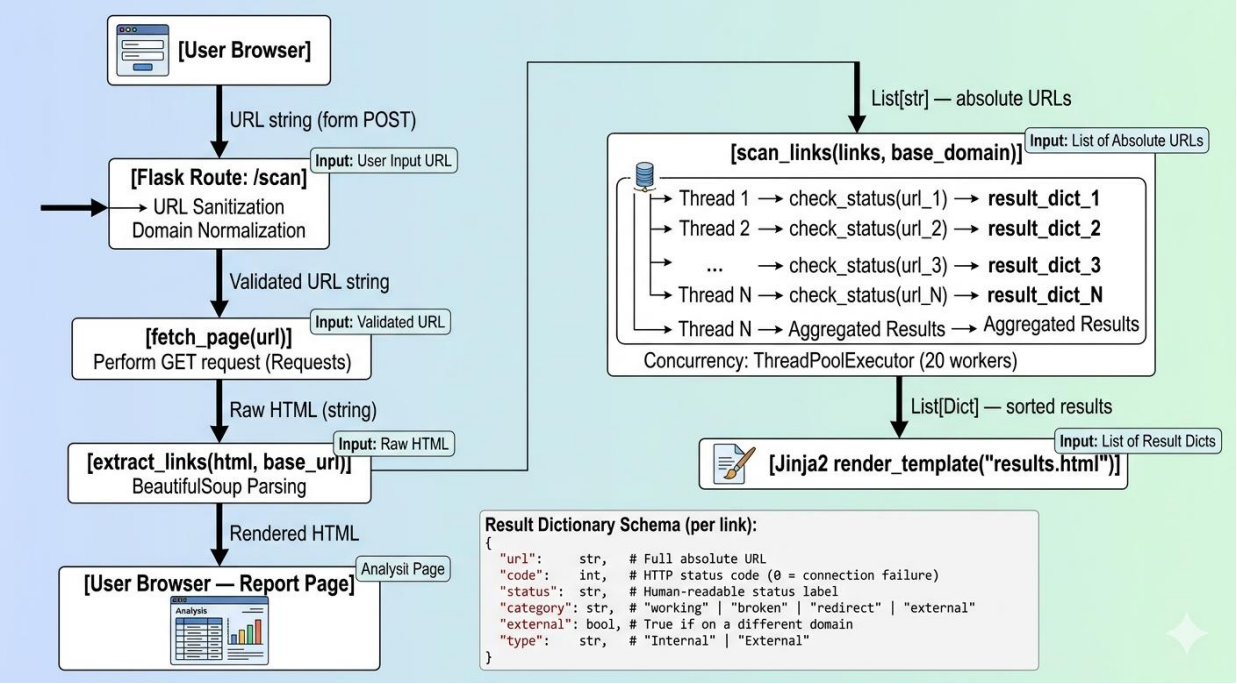
The execution follows a strict linear sequence:

- **User Submission:** The process starts when a user submits a URL via a JavaScript-enhanced web form, triggering a loading overlay for immediate feedback.
- **Request Handling:** The Flask backend receives the **HTTP POST** request and performs URL normalization and validation to ensure the target is reachable.
- **Initial Crawl:** The server uses the **Requests** library to fetch the page content and **BeautifulSoup** to parse and extract all embedded hyperlinks.
- **Parallel Verification:** To maximize performance, the system uses a **ThreadPoolExecutor (20 workers)** to perform simultaneous **HEAD requests**, checking the status code of every link without downloading full page data.
- **Intelligent Sorting:** Discovered links are automatically sorted by severity—prioritizing **broken (404)** and **redirected** links to highlight potential security risks.
- **Dynamic Reporting:** The final dataset is injected into a **Jinja2 template**, rendering a comprehensive results table back to the user's browser.



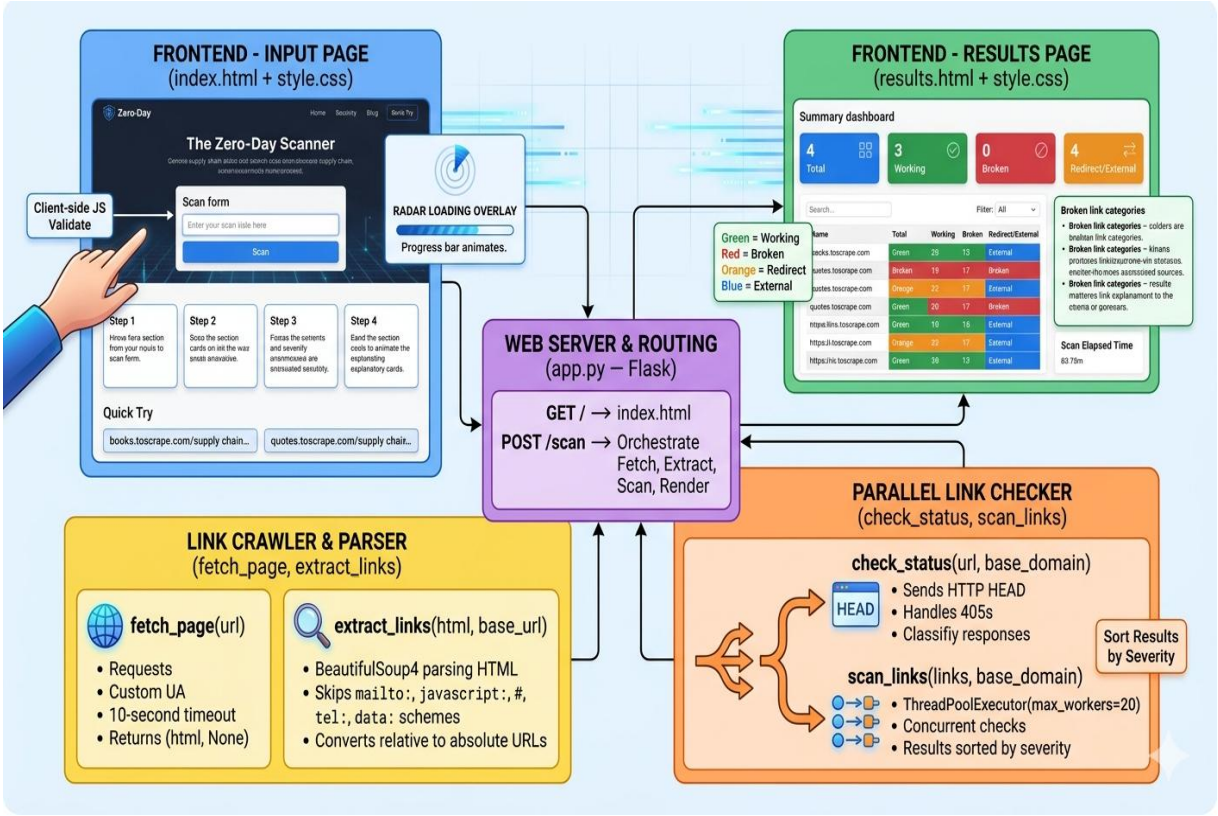
2.3. INFORMATION FLOW

- **Linear Transformation:** Converts a raw URL string into a structured data report through a step-by-step pipeline.
- **Data Sanitization:** The /scan route ensures only normalized URLs enter the system to prevent processing errors.
- **Extraction & Filtering:** extract_links strips away HTML noise, isolating only absolute URLs for analysis.
- **Concurrency (Fan-out):** scan_links distributes the URL list across 20 parallel threads for rapid status verification.
- **Structured Schema:** Each result is mapped to a strict dictionary (Schema) containing status codes, categories, and domain types.
- **UI Mapping:** The final sorted list is injected into **Jinja2**, transforming raw backend data into a visual HTML report.



2.4. COMPONENT DESIGN

- **Modular Architecture:** Splits the system into five specialized units, separating the user interface from the heavy backend logic.
- **User-Centric Frontend:** Includes an **Input Page** with real-time "radar" feedback and a **Results Page** featuring a color-coded summary dashboard.
- **Central Orchestration:** `app.py` serves as the system "brain," managing the transitions between fetching, parsing, and scanning while handling potential errors.
- **Robust Crawler & Parser:** Employs custom User-Agents and 10-second timeouts for stable fetching, while filtering non-web schemes (like `mailto:`) during parsing.
- **High-Speed Verification:** Uses a **ThreadPoolExecutor** with 20 workers and optimized **HTTP HEAD** requests to verify hundreds of links in seconds.
- **Severity-Based Reporting:** Automatically prioritizes critical risks, ensuring broken links (404s) appear at the top of the user report.



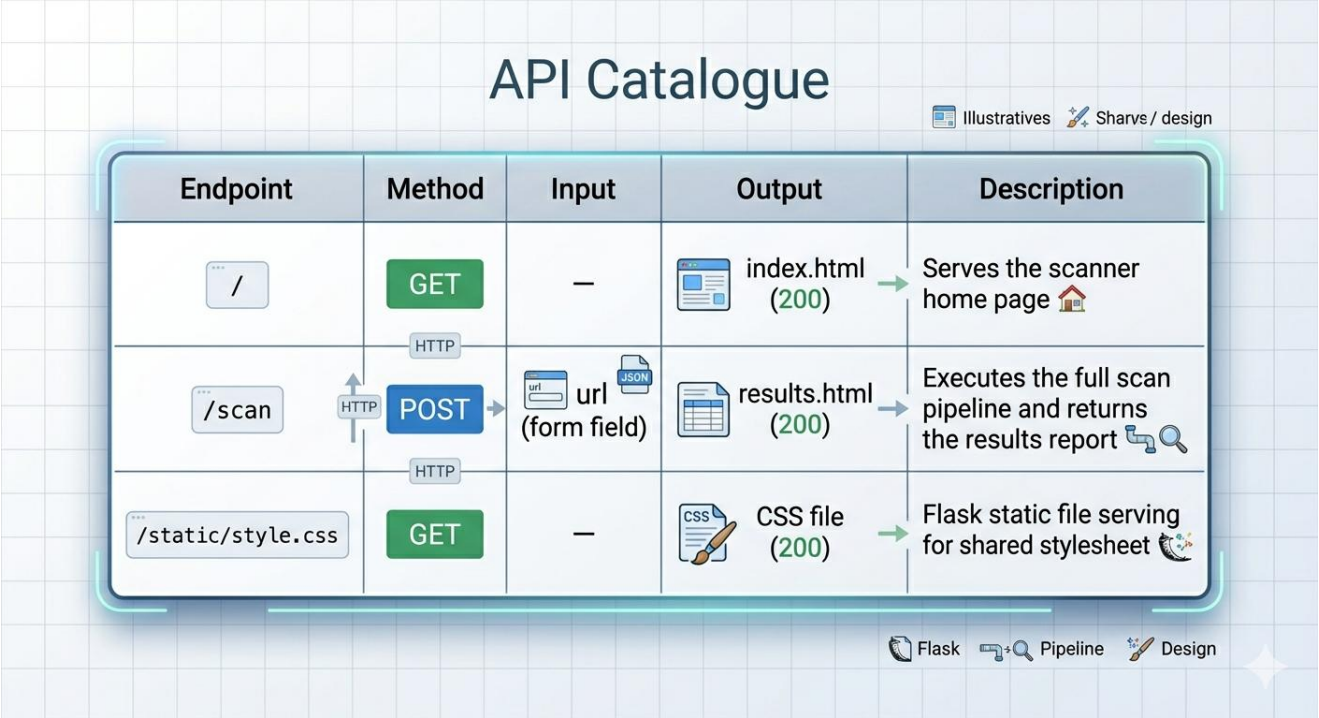
2.5. KEY DESIGN CONSIDERATIONS.

The system uses a browser-like User-Agent header while sending requests. This helps avoid blocking by servers that restrict requests from automated scripts.

- The system does not use a persistent database; instead, it stores results in memory since it is an educational tool and does not require data storage.
- Parallel scanning is implemented using ThreadPoolExecutor (20 threads) to significantly reduce scan time for multiple links.
- The system uses HEAD requests first for faster performance and falls back to GET if the server does not support HEAD.
- A custom User-Agent (ZeroDayScanner/1.0) is used to maintain transparency and inform server administrators about the tool's purpose.
- Server-side rendering is done using Flask and Jinja2 to keep the architecture simple and avoid CORS issues.
- The system automatically adds https:// to URLs if missing, improving user experience and reducing input errors.
- Errors are categorized into distinct types, allowing the frontend to display clear and helpful messages.
- Scan results are sorted by severity (Broken → Redirect → External → Working) so that critical issues are shown first.

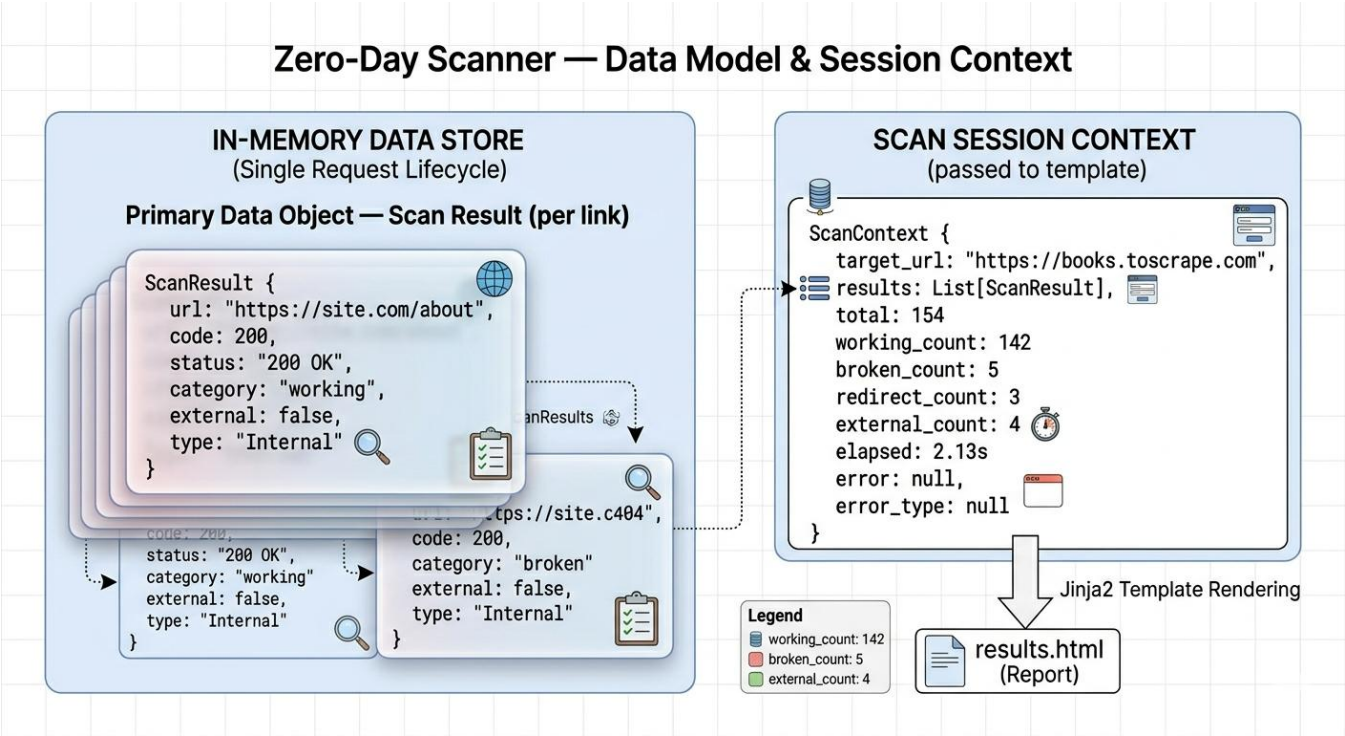
2.6. API CATALOGUE

The system exposes API endpoints are the following:



3.DATA DESIGN

3.1.DATA MODEL



3.2. DATA ACCESS MECHANISM

The frontend communicates with the backend using an HTML form submitted via POST with application/x-www-form-urlencoded encoding. The backend interacts with target websites using the Python requests library by sending HTTP GET and HEAD requests. Data is passed from the backend to the frontend using Flask's render_template() function, which injects Python dictionary data into Jinja2 templates for display. Static assets such as CSS files are served using Flask's built-in static file handler through the /static/ route.

The system does not use any ORM, database queries, or file input/output during normal operation.

3.3. DATA RETENTION POLICIES

The system does not retain scan results, as they are stored temporarily in memory and discarded after the HTTP response is completed. The user-submitted URL is used only during the current request and is not stored beyond that session. The HTML source of the target page is also processed in memory and discarded immediately after link extraction.
Log entries are only displayed in the console during runtime and are not permanently stored. Static assets such as HTML and CSS files are stored on the local filesystem as part of the project.
The system does not collect or store any personally identifiable information (PII).

3.4. DATA MIGRATION

- Data migration is not applicable in this version of the system because it does not use any persistent data storage.
- If the system is extended in the future, features like scan history would require a database such as SQLite or PostgreSQL, and user accounts would require a proper migration strategy using tools like Alembic with Flask-SQLAlchemy.

4.INTERFACES

- User Interface: A two-page architecture consisting of a high-engagement Home Page (with radar animations and demo buttons) and a data-rich Results Report (with a color-coded, filterable dashboard).
- External Boundaries: The system manages three primary outbound connections:
  1. Target Website: To fetch HTML content via HTTP GET.
  2. Discovered Links: To verify status codes via parallel HTTP HEAD requests.
  3. Google Fonts CDN: To deliver a professional typography suite (Inter and JetBrains Mono) directly to the browser.
- Internal Logic Flow: The backend utilizes a strict pipeline where the Flask Route orchestrates the sequence of fetch $\rightarrow$ extract $\rightarrow$ parallel scan before delivering the final context to the Jinja2 rendering engine.
- Design Standards: The UI is built using Semantic HTML5 for accessibility and a Responsive Grid to ensure cross-device compatibility.

Table 1: User Interface (UI) - Pages & Features

Page	URL	Key Features & Applied UI Principles
Home / Scan Input	http://127.0.0.1:5000/	Hero section, animated scan form, step-by-step explanation cards, quick-try demo butt. Radar loading animation, pulsing dots, particle effects. Accessibility (semantic HTML5, aria-hidden)
Scan Results / Report	http://127.0.0.1:5000/scan	Summary statistics dashboard (Counters), color-coded results table, detailed category explanations, scan time, error states. Colour Coding (Green : working, Red : broken, Orange : redirect, Blue : external) Responsive design

Colour Coding
Green : working
Red : broken
Orange : redirect
Blue : external

Animations

Table 2: External Interfaces - Protocols & Purpose

Interface	Protocol	Direction	Purpose
Target Website	HTTP/HTTPS (GET)	Outbound from server	Fetch the page HTML to crawl and parse
Discovered Links	HTTP/HTTPS (HEAD/GET)	Outbound from server	Check status of each link in parallel (HEAD falling back to GET on 405)
Google Fonts CDN	HTTPS	Outbound from browser	Load Inter (UI text) and JetBrains Mono (URL/code display) fonts

Table 2: External Interfaces - Protocols & Purpose

Interface	Protocol	Direction	Purpose
Target Website	HTTP/HTTPS (GET)	Outbound from server	Fetch the page HTML to crawl and parse.
Discovered Links	HTTP/HTTPS (HEAD/GET)	Outbound from server	Check status of each link in parallel (HEAD falling back to GET on 405).
Google Fonts CDN	HTTPS	Outbound from browser	Load Inter (UI text) and JetBrains Mono (URL/code display) fonts.

3: System Interface (Internal) - Orchestration Pipeline

Component	Action / Sub-component	Input / Output
Browser	POST /scan	Validated URL string / 200 HTML Report
Flask Route (/scan)	orchestrates fetch, extract, scan, render pipeline, handles error states	Validated URL string
fetch_page()	Perform HTTP GET, Custom User-Agent, 10-second timeout	Validated URL / (html, None) or (None, error_msg)
extract_links()	BeautifulSoup4 parsing HTML, Filter schemes (mailto, javascript, etc.)	html, base_url / List[str] absolute URLs
scan_links()	ThreadPoolExecutor(max_workers=20) concurrent checks, sorts results, links	base_domain / List[Dict] sorted results
render_template ("results.html")	Jinja2 rendering	ScanContext / Rendered HTML

5.STATE AND SESSION MANAGEMENT

- The Zero-Day Scanner is a stateless application where each scan is an independent HTTP request-response cycle.
- The server does not maintain any state, as Flask processes each request separately.
- The client side uses only minimal JavaScript state, such as controlling the loading overlay and progress bar.
- The system does not use session data or cookies.
- No authentication is required since it is designed as a single-user local tool.
- Refreshing the results page triggers a new request, and no previous scan history is stored.
- Future enhancements like multi-user support or scan history would require session management and database integration.

6.CACHING

In version 1.0, the system does not implement any explicit caching at any layer to ensure that all scan results are fresh and accurate. The Flask backend does not use caching, as storing previous results could lead to outdated or misleading information. The HTTP client (requests) also does not cache responses by default.

However, the browser caches static assets such as CSS files based on standard cache-control behavior, and DNS resolution benefits from OS-level caching.

In future versions, a short-lived in-memory caching mechanism (such as lru_cache or TTLCache) could be introduced to temporarily store scan results and reduce redundant network requests for repeated scans.

7.NON-FUNCTIONAL REQUIREMENTS

5.1.SECURITY ASPECTS

The system ensures ethical scanning by using a custom User-Agent to identify itself to target servers. It prevents injection attacks by validating user inputs and does not store any credentials or personal data, as all results are discarded after each scan. XSS is prevented through Jinja2's automatic escaping, and the system is designed with awareness of SSRF risks, though additional protections can be added in production. Rate limiting is not implemented in this version, and users are reminded to obtain permission before scanning any website.

5.2. PERFORMANCE ASPECTS

- The system improves performance by using parallel link checking with ThreadPoolExecutor, allowing multiple links to be scanned simultaneously and reducing total scan time. It uses HEAD requests before GET to minimize bandwidth and latency, and applies timeouts to prevent delays from slow or unresponsive targets.
- Since the system uses in-memory data, there is no database overhead, and static assets are efficiently served to reduce page load time. An animated progress bar provides user feedback during scanning, improving perceived performance. Typically, scans take around 5–30 seconds depending on the number of links and network conditions.
- However, the system is not designed for multi-user scalability, and production deployment would require tools like Gunicorn or uWSGI for better performance handling.

8.REFERENCES

- Flask Documentation — https://flask.palletsprojects.com/en/3.0.x/
- Python Requests Library — https://docs.python-requests.org/en/latest/
- BeautifulSoup4 Documentation — https://www.crummy.com/software/BeautifulSoup/bs4/doc/
- Python ThreadPoolExecutor — https://docs.python.org/3/library/concurrent.futures.html
- Jinja2 Template Engine — https://jinja.palletsprojects.com/en/3.1.x/
- OWASP Supply Chain Security — https://owasp.org/www-project-software-component-verification-standard/
- NIST SP 800-161: Cybersecurity Supply Chain Risk Management — https://csrc.nist.gov/publications/detail/sp/800-161/rev-1/final
- HTTP Status Codes (RFC 9110) — https://httpwg.org/specs/rfc9110.html
- Google Fonts (Inter & JetBrains Mono) — https://fonts.google.com
- MDN Web Docs — HTML5 Semantic Elements — https://developer.mozilla.org/en-US/docs/Web/HTML